

النوع List: مدخل إلى القوائم

11

النوع `list` (القائمة) هي بنية بيانات في بايثون، وهي عبارة عن تسلسل مُرتَّب قابل

للتغيير من تجميعية عناصر. سنتعرف في هذا الفصل على القوائم وتوابعها وكيفية استخدامها. القوائم مناسبة لتجميع العناصر المترابطة، إذ تمكّنك من تجميع البيانات المتشابهة، أو التي تخدم غرضًا معيّنًا معًا، وترتيب الشيفرة البرمجية، وتنفيذ التوابع والعمليات على عدة قيم في وقت واحد.

تساعد القوائم في بايثون وغيرها من هياكل البيانات المركبة على تمثيل التجميعات، مثل تجميعة ملفات في مجلد على حاسوبك، أو قوائم التشغيل، أو رسائل البريد الإلكتروني وغير ذلك.

في المثال التالي، سننشئ قائمةً تحتوي على عناصر من نوع السلاسل النصية:

```
sea_creatures = ['shark', 'cuttlefish', 'squid', 'mantis shrimp', 'anemone']
```

عندما نطبع القائمة، فستشبه المخرجات القائمة التي أنشأناها:

```
print(sea_creatures)
```

الناتج:

```
['shark', 'cuttlefish', 'squid', 'mantis shrimp', 'anemone']
```

بوصفها تسلسلاً مرتّبًا من العناصر، يمكن استدعاء أيّ عنصر من القائمة عبر الفهرسة.

القوائم هي بيانات مركّبة تتألف من أجزاء أصغر، وتتميز بالمرونة، إذ يمكن إضافة عناصر إليها، وإزالتها، وتغييرها. عندما تحتاج إلى تخزين الكثير من القيم، أو التكرار (`iterate`) عليها، وتريد أن تملك القدرة على تعديل تلك القيم بسهولة، فالقوائم هي خيارك الأفضل.

1. فهرسة القوائم (Indexing Lists)

كل عنصر في القائمة يقابله رقم يمثل فهرس ذلك العنصر، والذي هو عدد صحيح، فهرس العنصر الأول في القائمة هو 0.

إليك تمثيل لفهارس القائمة `sea_creatures`:

shark	shark	squid	shrimp	anemone
0	1	2	3	4

يبدأ العنصر الأول، أي السلسلة النصية `shark`، عند الفهرس 0، وتنتهي القائمة عند الفهرس 4 الذي يقابل العنصر `anemone`.

نظرًا لأن كل عنصر في قوائم بايثون يقابله رقم فهرس، يمكننا الوصول إلى عناصر القوائم ومعالجتها كما نفعل مع أنواع البيانات المتسلسلة الأخرى. يمكننا الآن استدعاء عنصر من القائمة من خلال رقم فهرسه:

```
print(sea_creatures[1]) # cuttlefish
```

تتراوح أرقام الفهارس في هذه القائمة بين 0 و 4، كما هو موضح في الجدول أعلاه. المثال التالي يوضح ذلك:

```
sea_creatures[0] = 'shark'
sea_creatures[1] = 'cuttlefish'
sea_creatures[2] = 'squid'
sea_creatures[3] = 'mantis shrimp'
sea_creatures[4] = 'anemone'
```

إذا استدعينا القائمة `sea_creatures` برقم فهرس أكبر من 4، فسيكون الفهرس خارج

النطاق، وسيطّلق الخطأ `IndexError`:

```
print(sea_creatures[18])
```

والمخرجات ستكون:

```
IndexError: list index out of range
```

يمكننا أيضًا الوصول إلى عناصر القائمة بفهارس سالبة، والتي تُحسب من نهاية القائمة،

بدءًا من -1. هذا مفيد في حال كانت لدينا قائمة طويلة، وأردنا تحديد عنصر في نهايته.

بالنسبة للقائمة `sea_creatures`، تبدو الفهارس السالبة كما يلي:

anemone	shrimp	squid	cuttlefish	shark
-1	-2	-3	-4	-5

في المثال التالي، سنطبع العنصر `squid` باستخدام فهرس سالب:

```
print(sea_creatures[-3]) # squid
```

يمكننا ضم (concatenate) سلسلة نصية مع سلسلة نصية أخرى باستخدام العامل +:

```
print('Sammy is a ' + sea_creatures[0]) # Sammy is a shark
```

لقد ضمنا السلسلة النصية `Sammy is a` مع العنصر ذي الفهرس 0. يمكننا أيضًا استخدام

المعامل + لضمّ قائمتين أو أكثر معًا (انظر الفقرة أدناه).

تساعدنا الفهارس على الوصول إلى أيّ عنصر من عناصر القائمة والعمل عليه.

2. تعديل عناصر القائمة

يمكننا استخدام الفهرسة لتغيير عناصر القائمة، عن طريق إسناد قيمة إلى عُنصر مُفهرس من القائمة. هذا يجعل القوائم أكثر مرونة، ويسهل تعديل وتحديث عناصرها.

إذا أردنا تغيير قيمة السلسلة النصية للعنصر الموجود عند الفهرس 1، من القيمة cuttlefish إلى octopus، فيمكننا القيام بذلك على النحو التالي:

```
sea_creatures[1] = 'octopus'
```

الآن، عندما نطبع sea_creatures، ستكون النتيجة:

```
print(sea_creatures)    # ['shark', 'octopus', 'squid', 'mantis
                        shrimp', 'anemone']
```

يمكننا أيضًا تغيير قيمة عنصر باستخدام فهرس سالب:

```
sea_creatures[-3] = 'blobfish'
print(sea_creatures)    # ['shark', 'octopus', 'blobfish',
                        'mantis shrimp', 'anemone']
```

الآن استبدلنا بالسلسلة squid السلسلة النصية blobfish الموجودة عند الفهرس

السالب 3- (والذي يقابل الفهرس الموجب 2).

3. تقطيع القوائم (Slicing Lists)

يمكننا أيضًا استدعاء عدة عناصر من القائمة. لنفترض أننا نرغب في طباعة العناصر

الموجودة في وسط القائمة sea_creatures، يمكننا القيام بذلك عن طريق اقتطاع شريحة (جزء) من القائمة.

يمكننا باستخدام الشرائح استدعاء عدة قيم عن طريق إنشاء مجال من الفهارس مفصولة

بنقطتين [x: y]:

```
print(sea_creatures[1:4])    # ['octopus', 'blobfish', 'mantis shrimp']
```

عند إنشاء شريحة، كما في [1: 4]، يبدأ الاقتطاع من العنصر ذي الفهرس الأول (مشمولاً)، وينتهي عند العنصر ذي الفهرس الثاني (غير مشمول)، لهذا طُبعت في مثالنا أعلاه العناصر الموجودة في المواضع، 1، و 2، و 3.

إذا أردنا تضمين أحد طرفي القائمة، فيمكننا حذف أحد الفهرسين في التعبير [x: y]. على سبيل المثال، إذا أردنا طباعة العناصر الثلاثة الأولى من القائمة sea_creatures، يمكننا فعل ذلك عن طريق كتابة:

```
print(sea_creatures[:3])    # ['shark', 'octopus', 'blobfish']
```

لقد طُبعت العناصر من بداية القائمة حتى العنصر ذي الفهرس 3. لتضمين جميع العناصر الموجودة في نهاية القائمة، سنعكس الصياغة:

```
print(sea_creatures[2:])    # ['blobfish', 'mantis shrimp', 'anemone']
```

يمكننا أيضاً استخدام الفهارس السالبة عند اقتطاع القوائم، تمامًا كما هو الحال مع

الفهارس الموجبة:

```
print(sea_creatures[-4:-2])    # ['octopus', 'blobfish']
print(sea_creatures[-3:])      # ['blobfish', 'mantis shrimp', 'anemone']
```

هناك معامل آخر يمكننا استخدامه في الاقتطاع، ويشير إلى عدد العناصر التي يجب أن تخطيها (الخطوة) بعد استرداد العنصر الأول من القائمة. حتى الآن، لقد أغفلنا المعامل `stride`، وستعطيه بايثون القيمة الافتراضية 1، ما يعني أنه سيتم استرداد كل العناصر الموجودة بين الفهرسين المحددين.

ستكون الصياغة على الشكل التالي `[x: y: z]`، إذ يشير `z` إلى الخطوة (`stride`). في المثال التالي، سننشئ قائمة كبيرة، ثم نقتطعها، مع خطوة اقتطاع تساوي 2:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

print(numbers[1:11:2])          # [1, 3, 5, 7, 9]
```

سيطبع التعبير `numbers[1: 11: 2]` القيم ذات الفهارس المحصورة بين 1 (مشمولة) و 11 (غير مشمولة)، وسيقفز البرنامج بخطوتين كل مرة، ويطبع العناصر المقابلة. يمكننا حذف المُعاملين الأوليين، واستخدام الخطوة وحدها بعدّها معاملاً وفق الصياغة التالية: `[::z]`:

```
print(numbers[::-3])          # [0, 3, 6, 9, 12]
```

عند طباعة القائمة `numbers` مع تعيين الخطوة عند القيمة 3، فلن تُطَبَّع إلا العناصر التي فهارسها من مضاعفات 3. يجعل استخدام الفهارس الموجبة والسالبة ومعامل الخطوة في اقتطاع القوائم التحكم في القوائم ومعالجتها أسهل وأكثر مرونة.

4. تعديل القوائم بالعوامل

يمكن استخدام العوامل لإجراء تعديلات على القوائم. سننظر في استخدام العاملين + و * ومقابليهما المركبين += و *=.

يمكن استخدام العامل + لضمّ (concatenate) قائمتين أو أكثر معًا:

```
sea_creatures = ['shark', 'octopus', 'blobfish', 'mantis
shrimp', 'anemone']
oceans = ['Pacific', 'Atlantic', 'Indian', 'Southern',
'Arctic']

print(sea_creatures + oceans)
```

والمخرجات هي:

```
['shark', 'octopus', 'blobfish', 'mantis shrimp', 'anemone',
'Pacific', 'Atlantic', 'Indian', 'Southern', 'Arctic']
```

يمكن استخدام العامل + لإضافة عنصر (أو عدة عناصر) إلى نهاية القائمة، لكن تذكر أن تضع العنصر بين قوسين مربعين:

```
sea_creatures = sea_creatures + ['yeti crab']
print (sea_creatures)      # ['shark', 'octopus', 'blobfish',
'mantis shrimp', 'anemone', 'yeti crab']
```

يمكن استخدام العامل * لمضاعفة القوائم (multiply lists). ربما تحتاج إلى عمل نُسخٍ لجميع الملفات الموجودة في مجلد على خادم، أو مشاركة قائمة أفلام مع الأصدقاء؛ ستحتاج في هذه الحالات إلى مضاعفة مجموعات البيانات.

سنضاعف القائمة sea_creatures مرتين، والقائمة oceans ثلاث مرات:


```
print(sea_creatures * 2)
print(oceans * 3)
```

والنتيجة ستكون:

```
['shark', 'octopus', 'blobfish', 'mantis shrimp', 'anemone',
'yeti crab', 'shark', 'octopus', 'blobfish', 'mantis shrimp',
'anemone', 'yeti crab']
['Pacific', 'Atlantic', 'Indian', 'Southern', 'Arctic',
'Pacific', 'Atlantic', 'Indian', 'Southern', 'Arctic',
'Pacific', 'Atlantic', 'Indian', 'Southern', 'Arctic']
```

يمكننا باستخدام العامل * نسخ القوائم عدة مرات.

يمكننا أيضًا استخدام الشكليات المركبة للعاملين + و * مع عامل الإسناد =. يمكن استخدام

العاملين المركبين += و *= لملء القوائم بطريقة سريعة ومؤتمتة. يمكنك استخدام هذين

العاملين لملء القوائم بعناصر نائبة (placeholders) يمكنك تعديلها في وقت لاحق بالمدخلات

المقدمة من المستخدم على سبيل المثال.

في المثال التالي، سنضيف عنصرًا إلى القائمة sea_creatures. سيعمل هذا العنصر مثل

عمل العنصر النائب، ونود إضافة هذا العنصر النائب عدة مرات. لفعل ذلك، سنستخدم العامل +=

مع الحلقة for.

```
for x in range(1,4):
    sea_creatures += ['fish']
print(sea_creatures)
```

والمخرجات ستكون:

```
['shark', 'octopus', 'blobfish', 'mantis shrimp', 'anemone',
'yeti crab', 'fish']
['shark', 'octopus', 'blobfish', 'mantis shrimp', 'anemone',
'yeti crab', 'fish', 'fish']
['shark', 'octopus', 'blobfish', 'mantis shrimp', 'anemone',
'yeti crab', 'fish', 'fish', 'fish']
```

سيُضاف لكل تكرار في الحلقة for عنصر fish إلى القائمة sea_creatures.

يتصرف العامل *= بطريقة مماثلة:

```
sharks = ['shark']

for x in range(1,4):
    sharks *= 2
    print(sharks)
```

النتائج سيكون:

```
['shark', 'shark']
['shark', 'shark', 'shark', 'shark']
['shark', 'shark', 'shark', 'shark', 'shark', 'shark', 'shark',
'shark']
```

5. إزالة عنصر من قائمة

يمكن إزالة العناصر من القوائم باستخدام del. سيؤدي ذلك إلى حذف العنصر الموجود عند

الفهرس المحدد.

سنزيل من القائمة sea_creatures العنصر octopus. هذا العنصر موجود عند الفهرس 1.

لإزالة هذا العنصر، سنستخدم del ثم نستدعي متغير القائمة وفهرس ذلك العنصر:

```
sea_creatures = ['shark', 'octopus', 'blobfish', 'mantis
shrimp', 'anemone', 'yeti crab']

del sea_creatures[1]
print(sea_creatures)      # ['shark', 'blobfish', 'mantis
shrimp', 'anemone', 'yeti crab']
```

الآن، العنصر ذو الفهرس 1، أي السلسلة النصية octopus، لم يعد موجودًا في قائمتنا. يمكننا أيضًا تحديد مجال مع العبارة del. لنقل أننا نريد إزالة العناصر octopus و blobfish و mantis shrimp معًا. يمكننا فعل ذلك على النحو التالي:

```
sea_creatures = ['shark', 'octopus', 'blobfish', 'mantis
shrimp', 'anemone', 'yeti crab']

del sea_creatures[1:4]
print(sea_creatures)      # ['shark', 'anemone', 'yeti crab']
```

باستخدام مجال مع del، تمكّننا من إزالة العناصر الموجودة بين الفهرسين 1 (مشمول) و 4 (غير مشمول)، والقائمة أضحت مكوّنة من 3 عناصر فقط بعد إزالة 3 عناصر منها.

6. بناء قوائم من قوائم أخرى موجودة

يمكن أن تتضمّن عناصر القوائم قوائم أخرى، مع إدراج كل قائمة بين قوسين معقوفين داخل الأقواس المعقوفة الخارجية التابعة لقائمة الأصلية:

```
sea_names = [['shark', 'octopus', 'squid', 'mantis shrimp'],
['Sammy', 'Jesse', 'Drew', 'Jamie']]
```

تسمى القوائم المُتضمّنة داخل قوائم أخرى بالقوائم المتشعبة (nested lists). للوصول إلى عنصر ضمن هذه القائمة، سيتعيّن علينا استخدام فهارس متعددة تقابل

مستوى التشعب:

```
print(sea_names[1][0])    # Sammy
print(sea_names[0][0])    # shark
```

فهرس القائمة الأولى يساوي 0، والقائمة الثانية فهرسها 1. ضمن كل قائمة متشعبة داخلية،

سيكون هناك فهرس منفصلة، والتي سنسميها فهرس ثانوية:

```
sea_names[0][0] = 'shark'
sea_names[0][1] = 'octopus'
sea_names[0][2] = 'squid'
sea_names[0][3] = 'mantis shrimp'

sea_names[1][0] = 'Sammy'
sea_names[1][1] = 'Jesse'
sea_names[1][2] = 'Drew'
sea_names[1][3] = 'Jamie'
```

عند العمل مع قوائم مؤلفة من قوائم، من المهم أن تعي أنك ستحتاج إلى استخدام أكثر من

فهرس واحد للوصول إلى عناصر القوائم المتشعبة.

7. استخدام توابع القوائم

سنتعرف الآن على التوابع المضمنة التي يمكن استخدامها مع القوائم. ونتعلم كيفية إضافة

عناصر إلى القائمة وكيفية إزالتها، وتوسيع القوائم وترتيبها، وغير ذلك.

القوائم أنواع قابلة للتغيير (mutable) على عكس السلاسل النصية التي لا يمكن تغييرها،

فعندما تستخدم تابعًا على قائمة ما، ستؤثر في القائمة نفسها، وليس في نسخة منها.

سنعمل في هذا القسم على قائمة تمثل حوض سمك، إذ ستحتوي القائمة أسماء أنواع

الأسماك الموجودة في الحوض، وسنعدلها كلما أضفنا أسماكًا أو أزلناها من الحوض.

١. التابع list.append()

يضيف التابع `list.append(x)` عنصرًا (العنصر `x` الممرّر) إلى نهاية القائمة `list`. يُعرّف المثال التالي قائمةً تمثل الأسماك الموجودة في حوض السمك.

```
fish = ['barracuda', 'cod', 'devil ray', 'eel']
```

تتألف هذه القائمة من 4 سلاسل نصية، وتتراوح فهرسها من 0 إلى 3. سنضيف سمكة جديدة إلى الحوض، ونود بالمقابل أن نضيف تلك السمكة إلى قائمتنا. سنمرّر السلسلة النصية `flounder` التي تمثل نوع السمكة الجديدة إلى التابع `list.append()`، ثم نطبع قائمتنا المعدلة لتأكيد إضافة العنصر:

```
fish.append('flounder')
print(fish)
# ['barracuda', 'cod', 'devil ray', 'eel', 'flounder']
```

الآن، صارت لدينا قائمة من 5 عناصر، تنتهي بالعنصر الذي أضفناه للتو عبر

التابع `append()`.

ب. التابع list.insert()

يأخذ التابع `list.insert(i,x)` وسيطين: الأول `i` يمثل الفهرس الذي ترغب في إضافة العنصر عنده، و `x` يمثل العنصر نفسه.

لقد أضفنا إلى حوض السمك سمكة جديدة من نوع `anchovy`. ربما لاحظت أنّ قائمة الأسماك مرتبة ترتيبًا أبجديًا حتى الآن، لهذا السبب لا نريد إفساد الترتيب، ولن نضيف السلسلة النصية `anchovy` إلى نهاية القائمة باستخدام الدالة `list.append()`؛ بدلًا من ذلك، سنستخدم التابع `list.insert()` لإضافة `anchovy` إلى بداية القائمة، أي عند الفهرس 0:

```
fish.insert(0, 'anchovy')
print(fish)
# ['anchovy', 'barracuda', 'cod', 'devil ray', 'eel',
  'flounder']
```

في هذه الحالة، أضفنا العنصر إلى بداية القائمة.

ستتقدم فهارس العناصر التالية خطوةً واحدةً إلى الأمام. لذلك، سيصبح العنصر

barracuda عند الفهرس 1، والعنصر cod عند الفهرس 2، والعنصر flounder - الأخير - عند الفهرس 5.

سنحضر الآن سمكة من نوع damselfish إلى الحوض، ونرغب في الحفاظ على الترتيب

الأبجدي لعناصر القائمة أعلاه، لذلك سنضع هذا العنصر عند الفهرس 3:

```
fish.insert(3, 'damselfish')
```

ج. التابع list.extend()

إذا أردت أن توسّع قائمة بعناصر قائمة أخرى، فيمكنك استخدام التابع list.extend(L)،

والذي يأخذ قائمة (المعامل L) ويضيف عناصرها إلى القائمة list.

سنضع في الحوض أربعة أسماك جديدة. أنواع هذه الأسماك مجموعة معًا في

القائمة more_fish:

```
more_fish = ['goby', 'herring', 'ide', 'kissing gourami']
```

سنضيف الآن عناصر القائمة more_fish إلى قائمة الأسماك، ونطبع القائمة لتأكد من أنَّ

عناصر القائمة الثانية قد أضيفت إليها:

```
fish.extend(more_fish)
```

```
print(fish)
```

ستطبع بايثون القائمة التالية:

```
['anchovy', 'barracuda', 'cod', 'devil ray', 'eel', 'flounder',  
'goby', 'herring', 'ide', 'kissing gourami']
```

في هذه المرحلة، صارت القائمة fish تتألف من 10 عناصر.

د. التابع list.remove()

لإزالة عنصر من قائمة، استخدم التابع list.remove(x)، والذي يزيل أول عنصر من القائمة له القيمة المُمرَّرة x.

جاءت مجموعة من العلماء المحليين لزيارة الحوض، وسيجرون أبحاثًا عن النوع kissing gourami، وطلبوا استعارة السمكة kissing gourami، لذلك نود إزالة العنصر kissing gourami من القائمة لعكس هذا التغيير:

```
fish.remove('kissing gourami')  
print(fish)
```

والمخرجات ستكون:

```
['anchovy', 'barracuda', 'cod', 'devil ray', 'eel', 'flounder',  
'goby', 'herring', 'ide']
```

بعد استخدام التابع list.remove()، لم يعد العنصر kissing gourami موجودًا في القائمة.

في حال اخترت عنصرًا x غير موجود في القائمة ومُرَّته إلى التابع list.remove()، فسيُطلق الخطأ التالي:

```
ValueError: list.remove(x): x not in list
```

لن يزيل التابع `list.remove()` إلا أوّل عنصر تساوي قيمته قيمة العنصر المُمرَّر إلى التابع، لذلك إن كانت لدينا سمكتان من النوع `kissing gourami` في الحوض، وأعرنا إحداهما فقط للعلماء، فإنّ التعبير `fish.remove('kissing gourami')` لن يمحو إلا العنصر الأوّل المطابق فقط.

ه. التابع `list.pop()`

يعيد التابع `list.pop([i])` العنصر الموجود عند الفهرس المحدد من القائمة، ثم يزيل ذلك العنصر. تشير الأقواس المربعة حول `i` إلى أنّ هذا المعامل اختياري، لذا، إذا لم تحدد فهرسًا (كما في `fish.pop()`)، فسيُعاد العنصر الأخير ثم يُزال.

لقد أصبح حجم السمكة `devil ray` كبيرًا جدًّا، ولم يعد الحوض يسعها، ولحسن الحظ أنّ هناك حوض سمك في بلدة مجاورة يمكنه استيعابها. سنستخدم التابع `pop()`، ونمرر إليه العدد 3، الذي يساوي فهرس العنصر `devil ray`، بقصد إزالته من القائمة. بعد إعادة العنصر، سنؤكد من أننا أزلنا العنصر الصحيح.

```
print(fish.pop(3)) # devil ray
print(fish)
# ['anchovy', 'barracuda', 'cod', 'eel', 'flounder', 'goby',
  'herring', 'ide']
```

باستخدام التابع `pop()`، تمكّنّا من إزالة السمكة `devil ray` من قائمة الأسماك. إذا لم تُمرَّر أيّ معامل إلى هذا التابع، ونفّذنا الاستدعاء `fish.pop()`، فسيُعاد العنصر الأخير `ide` ثم يُزال من القائمة.

و. التابع list.index()

يصعب في القوائم الكبيرة تحديد فهرس العناصر التي تحمل قيمة معينة. لأجل ذلك، يمكننا استخدام التابع `list.index(x)`، إذ يمثل الوسيط `x` قيمة العنصر المبحوث عنه، والذي نريد معرفة فهرسه. إذا كان هناك أكثر من عنصر واحد يحمل القيمة `x`، فسيُعَاد فهرس العنصر الأول.

```
print(fish)      # ['anchovy', 'barracuda', 'cod', 'eel',
                  'flounder', 'goby', 'herring', 'ide']
```

```
print(fish.index('herring'))    # 6
```

سوف يُطْلَق خطأ في حال مرّرنا قيمة غير موجودة في القائمة إلى التابع `..index()`.

ز. التابع list.copy()

أحياناً نرغب في تعديل عناصر قائمة والتجريب عليها، مع الحفاظ على القائمة الأصلية دون تغيير؛ يمكننا في هذه الحالة استخدام التابع `list.copy()` لإنشاء نسخة من القائمة الأصلية. في المثال التالي، سنمرّر القيمة المعادة من `fish.copy()` إلى المتغير `fish_2`، ثم نطبع قيمة `fish_2` للتأكد من أنَّها تحتوي على نفس عناصر القائمة `fish`.

```
fish_2 = fish.copy()
print(fish_2)
# ['anchovy', 'barracuda', 'cod', 'eel', 'flounder', 'goby',
  'herring', 'ide']
```

في هذه المرحلة، القامتان `fish` و `fish_2` متساويتان.

ح. التابع `list.reverse()`

يمكننا عكس ترتيب عناصر قائمة باستخدام التابع `list.reverse()`. في المثال التالي

سنستخدم التابع `reverse()` مع القائمة `fish` لعكس ترتيب عناصرها.

```
fish.reverse()
print(fish)      # ['ide', 'herring', 'goby', 'flounder', 'eel',
                  'cod', 'barracuda', 'anchovy']
```

بعد استخدام التابع `reverse()`، صارت القائمة تبدأ بالعنصر `ide`، والذي كان في نهاية

القائمة من قبل، كما ستنتهي القائمة بالعنصر `anchovy`، والذي كان في بداية القائمة من قبل.

ط. التابع `list.count()`

يعيد التابع `list.count(x)` عدد مرات ظهور القيمة `x` في القائمة. هذا التابع مفيد في

حال كنا نعمل على قائمة طويلة بها الكثير من القيم المتطابقة.

إذا كان حوض السمك كبيرًا، على سبيل المثال، وكانت عندنا عدة أسماك من النوع

`neon tetra`، فيمكننا استخدام التابع `count()` لتحديد العدد الإجمالي لأسماك هذا النوع.

في المثال التالي، سنحسب عدد مرات ظهور العنصر `goby`:

```
print(fish.count('goby'))      # 1
```

تظهر السلسلة النصية `goby` مرةً واحدةً فقط في القائمة، لذا سيعيد التابع `count()`

العدد 1. يمكننا استخدام هذا التابع أيضًا مع قائمة مكوّنة من **أعداد صحيحة**، ويوضح المثال

التالي ذلك.

يتتبع المشرفون على الحوض أعمار الأسماك الموجودة فيه للتأكد من أنّ وجباتها الغذائية

مناسبة لأعمارها. هذه القائمة الثانية المُسمّاة `fish_ages` تتوافق مع أنواع السمك في

القائمة `fish`. نظرًا لأن الأسماك التي لا يتجاوز عمرها عامًا واحدًا لها احتياجات غذائية خاصة، فسنحسب عدد الأسماك التي عمرها عامًا واحدًا:

```
fish_ages = [1,2,4,3,2,1,1,2]
print(fish_ages.count(1))    # 3
```

يظهر العدد الصحيح 1 في القائمة `fish_ages` ثلاث مرات، لذلك يعيد التابع `count()` العدد 3.

ي. التابع `list.sort()`

يُستخدم التابع `list.sort()` لترتيب عناصر القائمة التي استُدعي معها. سنستخدم قائمة الأعداد الصحيحة `fish_ages` لتجريب التابع `sort()`:

```
fish_ages.sort()
print(fish_ages)    # [1, 1, 1, 2, 2, 2, 3, 4]
```

باستدعاء التابع `sort()` مع القائمة `fish_ages`، فستُعاد تلك القائمة مرتبةً.

ك. التابع `list.clear()`

بعد الانتهاء من العمل على قائمة ما، يمكنك إزالة جميع العناصر الموجودة فيها باستخدام التابع `list.clear()`.

قررت الحكومة المحلية الاستيلاء على حوض السمك الخاص بنا، وجعله مساحة عامة يستمتع بها سكان مدينتنا. نظرًا لأننا لم نعد نعمل على الحوض، فلم نعد بحاجة إلى الاحتفاظ بقائمة الأسماك، لذلك سنزيل عناصر القائمة `fish`:

```
fish.clear()
print(fish)    # []
```

نرى في المخرجات أقواسًا معقوفة نتيجة استدعاء التابع `clear()` على القائمة `fish`، وهذا تأكيد على أنَّ القائمة أصبحت خالية من جميع العناصر.

8. فهم كيفية استعمال List Comprehensions

توفر `List Comprehensions` طريقةً مختصرةً لإنشاء القوائم بناءً على قوائم موجودة مسبقًا. فيمكن عند استخدام `list comprehensions` بناء القوائم باستخدام أي نوع من البيانات المتسلسلة التي يمكن الدوران على عناصرها عبر حلقات التكرار، بما في ذلك السلاسل النصية والصفوف. من ناحية التركيب اللغوي، تحتوي `list comprehensions` على عنصر يمكن المرور عليه ضمن تعبير متبوع بحلقة `for`. ويمكن أن يُتبع ما سبق بتعابير `for` أو `if` إضافية، لذا سيساعدك الفهم العميق **لحلقات `for`** والعبارات الشرطية في التعامل مع `list comprehensions`. توفر `list comprehensions` طريقةً مختلفةً لإنشاء القوائم وغيرها من أنواع البيانات المتسلسلة. وعلى الرغم من إمكانية استخدام الطرائق الأخرى للدوران، مثل حلقات `for`، لإنشاء القوائم، لكن من المفضل استعمال `list comprehensions` لأنها تقلل عدد الأسطر الموجودة في برنامجك.

يمكن بناء `list comprehensions` في بايثون كالآتي:

```
list_variable = [x for x in iterable]
```

سُيُسَد القائمة، أو أي نوع من البيانات يمكن المرور على عناصره، إلى متغير. المتغيرات الإضافية -التي تُشير إلى عناصر موجودة ضمن نوع البيانات الذي يمكن المرور على عناصره-

تُبنى حول عبارة `for`. والكلمة المحجوزة `in` تستعمل بنفس استعمالها في حلقات `for` وذلك لمرور على عناصر `iterable`. لننظر إلى مثال يُنشئ قائمةً مبنيةً على سلسلة نصية:

```
shark_letters = [letter for letter in 'shark']
print(shark_letters)
```

أسندنا في المثال السابق قائمةً جديدةً إلى المتغير `shark_letters`، واستعملنا المتغير `letter` للإشارة إلى العناصر الموجودة ضمن السلسلة النصية `'shark'`. استعملنا بعد ذلك الدالة `print()` لكي نتأكد من القائمة الناتجة والمُسندة إلى المتغير `shark_letters`، وحصلنا على الناتج الآتي:

```
['s', 'h', 'a', 'r', 'k']
```

تتألف القائمة التي أنشأناها باستخدام `list comprehensions` من العناصر التي تكوّن السلسلة النصية `'shark'`، وهي كل حرف في الكلمة `shark`. يمكن إعادة كتابة تعابير `list comprehensions` بشكل حلقات `for`، لكن لاحظ أنك لا تستطيع إعادة كتابة كل حلقة `for` بصيغة `list comprehensions`. لنعد كتابة المثال السابق الذي أنشأنا فيه القائمة `shark_letters` باستخدام حلقة `for`، وهذا سيساعدنا في فهم كيف تعمل `list comprehensions` عملها:

```
shark_letters = []

for letter in 'shark':
    shark_letters.append(letter)

print(shark_letters)
```

عند إنشائنا للقائمة عبر استخدام الحلقة `for`، فيجب تهيئة المتغير الذي سُسند العناصر

إليه كقائمة فارغة، وهذا ما فعلناه في أول سطر من الشيفرة السابقة. ثم بدأت حلقة `for` بالدوران على عناصر السلسلة النصية `'shark'` مستعملًا المتغير `letter` للإشارة إلى قيمة العنصر الحالي، ومن ثم أضفنا كل عنصر في السلسلة النصية إلى القائمة ضمن حلقة `for` وذلك باستخدام الدالة `list.append(x)`. الناتج من حلقة `for` السابقة يماثل ناتج `list comprehension` في المثال أعلاه:

```
['s', 'h', 'a', 'r', 'k']
```

يمكن إعادة كتابة `List comprehensions` كحلقات `for`، لكن بعض حلقات `for` يمكن إعادة كتابتها لتصبح `List comprehensions` لتقليل كمية الشيفرات المكتوبة.

١. استخدام التعابير الشرطية مع `List Comprehensions`

يمكن استخدام التعابير الشرطية في `list comprehension` لتعديل القوائم أو أنواع البيانات المتسلسلة الأخرى عند إنشاء قوائم جديدة. لننظر إلى مثال عن استخدام العبارة الشرطية `if` في تعبير `list comprehension`:

```
fish_tuple = ('blowfish', 'clownfish', 'catfish', 'octopus')

fish_list = [fish for fish in fish_tuple if fish != 'octopus']
print(fish_list)
```

استعملنا المتغير `fish_tuple` الذي من نوع البيانات `tuple` أساسًا للقائمة الجديدة التي سننشئها التي تسمى `fish_list`. استعملنا `for` و `in` كما في القسم السابق، لكننا أضفنا هنا التعليمة الشرطية `if`. ستؤدي التعليمة الشرطية `if` إلى إضافة العناصر غير المساوية للسلسلة النصية `'octopus'`، لذا ستحتوي القائمة الجديدة على العناصر الموجودة في بنية صف

(tuple) والتي لا تُطابق الكلمة 'octopus'. عند تشغيل البرنامج السابق فسنلاحظ أنَّ القائمة fish_list تحتوي على نفس العناصر التي كانت موجودة في fish_tuple لكن مع حذف العنصر 'octopus':

```
['blowfish', 'clownfish', 'catfish']
```

أي أصبحت القائمة الجديدة تحتوي على بنية صف أصلية لكن ما عدا السلسلة النصية التي استثنيناها عبر التعبير الشرطي. سنُنشئ مثالاً آخر يستعمل المعاملات الرياضية والأرقام الصحيحة والدالة range():

```
number_list = [x ** 2 for x in range(10) if x % 2 == 0]
print(number_list)
```

القائمة التي سنُنشئ باسم number_list ستحتوي على مربع جميع القيم الموجودة من المجال 0 إلى 9 لكن إذا كان الرقم قابلاً للقسمة على 2. وستبدو المخرجات الآتية:

```
[0, 4, 16, 36, 64]
```

دعنا نُفَصِّل ما الذي يفعله تعبير list comprehension السابق، ودعنا نفكر بالذي سيظهر إذا استعملنا التعبير for x in range(10) فقط. يجب أن يبدو برنامجنا الصغير كالاتي:

```
number_list = [x for x in range(10)]
print(number_list)
```

الناتج:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

لنصف العبارة الشرطية الآن:

```
number_list = [x for x in range(10) if x % 2 == 0]
print(number_list)
```

النتيجة:

```
[0, 2, 4, 6, 8]
```

أدت التعليمة الشرطية `if` إلى قبول العناصر القابلة للقسمة على 2 فقط وإضافتها إلى القائمة، مما يؤدي إلى حذف جميع الأرقام الفردية. يمكننا الآن استخدام معامل رياضي لتربيع قيمة المتغير `x`:

```
number_list = [x ** 2 for x in range(10) if x % 2 == 0]
print(number_list)
```

أي سترجع قيم القائمة السابقة [0, 2, 4, 6, 8] وسيخرج الناتج الآتي:

```
[0, 4, 16, 36, 64]
```

يمكننا أيضًا استعمال ما يشبه عبارات `if` المتشعبة في تعابير `list comprehension`:

```
number_list = [x for x in range(100) if x % 3 == 0 if x % 5 == 0]
print(number_list)
```

سيتم التحقق أولاً أنَّ المتغير `x` قابل للقسمة على الرقم 3، ثم سنتحقق إن كان المتغير `x` قابل للقسمة على الرقم 5، وإذا حقق المتغير `x` الشرطين السابقين فسيُضاف إلى القائمة، وسيُظهر في الناتج:

```
[0, 15, 30, 45, 60, 75, 90]
```


يمكن استخدام تعليمة if الشرطية لتحديد ما هي العناصر التي نريد إضافتها إلى القائمة الجديدة.

ب. حلقات التكرار المتشعبة في تعابير List Comprehension

يمكن استعمال حلقات التكرار المتشعبة لإجراء عدّة عمليات دوران متداخلة في برامجنا. سننظر في هذا القسم إلى حلقة for متشعبة وسنحاول تحويلها إلى تعبير list comprehension. سننشئ هذه الشيفرة قائمةً جديدةً بالدوران على قائمتين وبإجراء عمليات رياضية عليها:

```
my_list = []

for x in [20, 40, 60]:
    for y in [2, 4, 6]:
        my_list.append(x * y)

print(my_list)
```

سنحصل على الناتج الآتي عند تشغيل البرنامج:

```
[40, 80, 120, 80, 160, 240, 120, 240, 360]
```

تضرب الشيفرة السابقة العناصر الموجودة في أوّل قائمة بالعناصر الموجودة في ثاني قائمة في كل دورة. لتحويل ما سبق إلى تعبير list comprehension، وذلك باختصار السطرين الموجودين في الشيفرة السابقة وتحويلهما إلى سطرٍ وحيدٍ، الذي يبدأ بإجراء العملية $x*y$ ، ثم ستلي هذه العملية حلقة for الخارجية، ثم يليها حلقة for الداخلية؛ وسنضيف تعبير print() للتأكد أنّ ناتج القائمة الجديدة يُطابق ناتج البرنامج الذي فيه حلقتين متداخلتين:

```
my_list = [x * y for x in [20, 40, 60] for y in [2, 4, 6]]
print(my_list)
```

الناتج:

```
[40, 80, 120, 80, 160, 240, 120, 240, 360]
```

أدى استعمال تعبير list comprehension في المثال السابق إلى تبسيط حلقتي for لتصبحا سطرًا واحدًا، لكن مع إنشاء نفس القائمة والتي سئسَد إلى المتغير my_list. توفر لنا تعابير list comprehension طريقةً بسيطةً لإنشاء القوائم، مما يسمح لنا باختصار عدّة أسطر إلى سطرٍ وحيد. لكن من المهم أن تبقى في ذهنك أنّ سهولة قراءة الشيفرة لها الأولوية دومًا، لذا إذا أصبحت تعابير list comprehension طويلةً جدًا ومعقّدة، فمن الأفضل حينها تحويلها إلى حلقات تكرار عادية.

9. خلاصة الفصل

القوائم هي نوع بيانات مرّن يمكن تعديله بسهولة خلال أطوار البرنامج. غطينا في هذا الفصل الميزات والخصائص الأساسية لقوائم، بما في ذلك الفهرسة والاقتطاع والتعديل والضمّ. لمّا كانت القوائم تسلسلات قابلة للتغيير (mutable)، فإنّها هياكل بيانات مرنة ومفيدة للغاية. كما تتيح لنا توابع القوائم إجراء العديد من العمليات على القوائم بسهولة، إذ يمكننا استخدام التوابع لتعديل القوائم وترتيبها ومعالجتها بفعالية.

تسمح تعابير list comprehension لنا بتحويل قائمة أو أي نوع من البيانات المتسلسلة إلى سلسلة جديدة، ولها شكلٌ بسيطٌ يُقلّل عدد الأسطر التي نكتبها. تتبع تعابير list comprehension شكلًا رياضيًا معيّنًا، لذا قد يجدها المبرمجون أولو الخلفية الرياضية سهولة الفهم. وصحيح أنّ

تعبير list comprehension تختصر الشيفرة. لكن من المهم جعل سهولة قراءة الشيفرة من أولوياتنا، وحاول تجنّب الأسطر الطويلة لتسهيل قراءة الشيفرة.